



GROUP PROJECT REPORT

COURSE CODE:

WIA1006 – MACHINE LEARNING

PROJECT TITLE:

**GhostBusters - Predicting Digital Disappearances with
Explainable AI**

LECTURER NAME:

DR. SHAPLA KHANAM

GROUP:

GROUP 6 (OCC 6)

GROUP MEMBERS:

NAME	MATRIX NUMBER
THET HTUN OAKAR	24209805
YUDONG CHEN	24074901
ABDURRAHMAN AHMAD	24083903
NUR ARINAH ALIYAH BINTI NASFARAZI	24001616
NUR FARAH IRDINA BINTI MOHD RIZAM	24001443
NUR ANITH RUWAIDA BINTI AMDAN	24001645

1. Introduction

In the era of constant connectivity, relationships are increasingly shaped by digital interactions. Conversations unfold through instant messages, emotions are conveyed via emojis, and silence can signal more than words. A defining phenomenon of modern digital dating is **ghosting**: the sudden, unexplained withdrawal of communication by one party, leaving the other without closure or explanation. As dating applications have become the primary venue for romantic initiation, ghosting has evolved from a fringe behaviour into a near-universal experience, with studies suggesting the majority of online daters have either been ghosted or have ghosted someone else.

This project, GhostBusters: Predicting Digital Disappearances with Explainable AI, is developed under the course theme “Tying the (Data) Knot: Love, Life & Likes” for WIA1006/WID3006 Machine Learning (Sem 2, 2025/2026). Using a synthetic dataset of 50,000 dating app user records sourced from Kaggle, the project applies supervised machine learning to predict whether a user will ghost their match, based on behavioural signals and demographic profile.

What distinguishes this project is the integration of **Explainable AI (XAI)** via SHAP (SHapley Additive exPlanations). Rather than producing opaque, black-box predictions, the system surfaces why a specific user is predicted to ghost, identifying which behavioural features are the strongest drivers. This transparency is critical for interpretability, trust, and any real-world application of such a system.

The full pipeline covers: data loading and preprocessing, feature engineering, exploratory data analysis (EDA), training and tuning of the five machine learning models (Logistic Regression, Decision Tree, Random Forest, SVM, and Neural Network/MLP), AutoML benchmarking against auto-sklearn, and SHAP-based feature importance analysis.

2. Problem Statement & Objectives

2.1 Problem Statement

Ghosting in digital dating environments presents both social and a computational challenge. Despite its prevalence, it remains difficult to anticipate, and even harder to explain, because the decision to ghost is shaped by a complex interplay of behavioural signals, psychological predispositions, and contextual factors embedded within app usage data.

Existing dating applications track rich streams of user interaction data, yet they do not leverage this data to detect or warn about impending disengagement. This represents an untapped opportunity: if a predictive model can identify users at high risk of ghosting before they go silent, platforms could intervene to improve match quality, communication prompts, or user experience, ultimately reducing emotional harm.

The core problem is therefore: *give a user's behavioural and demographic profile on a dating application, can we accurately predict whether that user will ghost their current match? If so, explain which factors contributed most strongly to that prediction?*

2.2 ML Problem Type

This project is framed as a Supervised Binary Classification task.

- **Target Variable:** The target variable is `is_ghosted`, derived from the original `match_outcome` column by isolating the “Ghosted” label against all other outcomes (e.g., Mutual Match, Catfished)
- **Positive Class:** Class 1 (Ghosted) represents the positive class, the outcome we aim to detect.
- **Negative Class:** Class 0 (Not Ghosted) represents all remaining outcomes, treating them as non-ghosting events.

2.3 Project Objectives

The project pursues the following objectives:

- **Objective 1:** Develop a robust binary classification pipeline that predicts whether a dating app user will ghost their match, trained on 50,000 records with 19 behavioural and demographic features.
- **Objective 2:** Train, compare, and evaluate five diverse machine learning models. Logistic Regression, Decision Tree, Random Forest, SVM (SDG-Hinge), and Neural Network (MLP) using accuracy, AUC-ROC, precision and recall.
- **Objective 3:** Address class imbalance through class-weight balancing and SMOTE oversampling, ensuring the minority class (ghosters) is adequately represented during model training.
- **Objective 4:** Optimise the Random Forest classifier using GridSearchCV with 3-fold cross-validation to identify the best hyperparameter configuration.
- **Objective 5:** Benchmark all human-tuned models against an AutoML baseline (auto-sklearn) to assess whether automated pipelines offer a competitive advantage.
- **Objective 6:** Apply SHAP (SHapely additive exPlanations) to the best-performing model to deliver feature-level explanations, qualifying each feature's contribution to individual predictions and global model behaviour.

2.4 Dataset & Feature Overview

The dataset used is the Dating App Behaviour Dataset (Kaggle), containing 50,000 synthetic user records with 19 features. These are categorised by their behavioural relevance to ghosting prediction as follows:

Feature Category	Examples	Why It Matters
Engagement Metrics	app_usage_time_min, message_sent_count	Proxy for user investment; low engagement often precedes ghosting.
User Temperament	swipe_right_ratio, emoji_usage_rate	Reflects pickiness and interaction style; high swipe ratios may signal choice overload and burnout.
Demographics	income_bracket, education_level	Socio-economic context that may shape social interaction styles and commitment patterns.
Social Proof	likes_received, mutual_matches	Indicates user popularity; high-choice environments have been linked to more frequent ghosting.
Temporal Patterns	last_active_hour, swipe_time_of_day	When a user is active may differentiate casual from intentional engagement.

2.5 Success Metrics

The following evaluation metrics are used to compare all models, selected to account for the class imbalance inherent in ghosting event prediction:

Metric	Why It Is Used
Accuracy	Overall correctness; useful as a sanity check but can be misleading under class imbalance.
F1-Score	Harmonic mean of Precision and Recall. The primary ranking metric, ensuring ghosters are identified without excessive false positives.
AUC-ROC	Measures the model’s ability to distinguish between classes across all decision thresholds.
Precision & Recall	Precision: of predicted ghosters, how many truly ghosted. Recall: of all actual ghosters, how many were caught.
SHAP Value Mean Magnitude	Quantifies each feature’s average contribution to predictions, used for post-hoc interpretability.

2.6 Expected Outcomes

Ensemble and neural network models (Random Forest, Neural Network/MLP) are expected to outperform linear baselines (Logistic Regression, SVM) due to the non-linear nature of human behaviour in digital dating environments.

Furthermore, behavioural engagement features (e.g., message_sent_count, swipe_right_ratio, app_usage_time_min) are hypothesised to be stronger predictors of ghosting than demographic features (e.g., income_bracket, education_level), as ghosting is more directly driven by in-app behaviour than by background characteristics. SHAP analysis will be used to verify or challenge these hypotheses after model training.

3. Data Description & Preprocessing Methodology

Building upon the problem objectives established in Section 2, this section delineates the technical execution of the data pipeline. To convert the 50,000 raw simulated records into an optimal format for training our five core models and the auto-sklearn meta-learner, data preparation was anchored entirely within a scikit-learn Pipeline architecture. This approach guarantees strict reproducibility and systematically prevents data leakage.

3.1 Data Description

The raw *Dating App Behaviour Dataset* contains 50,000 observations across 19 feature columns. While the behavioural context and relevance of these variables were established in Section 2.4, their baseline software data types must be explicitly mapped and managed before entering the machine learning pipeline:

- **Categorical / String Features (8):** gender, sexual_orientation, location_type, income_bracket, education_level, app_usage_time_label, swipe_right_label, swipe_time_of_day.
- **Numerical Continuous / Discrete Features (10):** .app_usage_time_min, swipe_right_ratio, likes_received, mutual_matches, profile_pics_count, bio_length, message_sent_count, emoji_usage_rate, last_active_hour, and the engineered interest_count attribute.

3.2 Preprocessing Methodology

3.2.1 Environment-Agnostic Data Loading

To enable collaboration across local Jupyter Notebook environments and Google Colab instances without requiring manual file path alterations, a dynamic directory resolution script was implemented:

```
import os

# Works on Google Colab AND Local - tries both paths
for path in [
    ... '/content/dating_app_behavior_dataset.csv',
    ... '/content/sample_data/dating_app_behavior_dataset.csv',
    ... 'dating_app_behavior_dataset.csv',
]:
    if os.path.exists(path):
        ... df = pd.read_csv(path)
        ... print(f'Dataset loaded from: {path}')
        ... print(f'Shape: {df.shape}')
        ... break
    else:
        ... raise FileNotFoundError(
        ...     "Upload 'dating_app_behavior_dataset.csv' to Colab or the same folder as this notebook."
        ... )
df.head()
```

Upon execution, the script verifies the dataset's baseline dimensions matrix of **50,000 rows and 19 columns**, ensuring that the full volume of simulated interactions is structurally accounted for before processing.

3.2.2 Target Variable Binarization

As outlined in the binary classification framework (Section 2.2), the multi-class string arrays from the raw `match_outcome` column must be binarized. The operational code maps the “Ghosted” class as the positive indicator (1) and pools all other stable interaction states as the negative baseline (0):

```
# Define target: Is the outcome 'Ghosted'?
# This turns it into a binary classification problem for the XAI model.
df['is_ghosted'] = (df['match_outcome'] == 'Ghosted').astype(int)
df = df.drop(columns=['match_outcome'])

print(f"Dataset shape: {df.shape}")
df.head()
```

- **Dimensionality Maintenance:** Dropping the original categorical `match_outcome` column immediately after creating `is_ghosted` keeps the total column count stable. This completely purges raw target components from the feature matrix, mitigating any risk of target data leakage.

3.2.3 Feature Engineering: Unstructured Text Transformation

The raw `interest_tags` variables consist of high-cardinality, comma-separated text strings listing individual user preferences (e.g., “Cooking, Hiking, Gaming”). Passing raw text fields presents a severe high_cardinality challenge for tabular machine learning models.

To capture the predictive signal of a user’s hobby density and profile completeness while keeping the data structured for Explainable AI (XAI) frameworks, the field was engineered into a continuous numerical attribute (`interest_count`).

```
# Count the number of interests listed
df['interest_count'] = df['interest_tags'].apply(lambda x: len(x.split(',')) if isinstance(x, str) else 0)

# Drop the raw text column for the primary model (XAI models prefer structured data)
df = df.drop(columns=['interest_tags'])
```

- **Methodology Rationale:** This transformation converts an uninterpretable text block into a discrete integer data type, allowing distance-based and tree-based split algorithms to identify threshold effects. An explicit string type-check (`isinstance(x, str)`) ensures that blank profiles safely default to a count of 0 instead of triggering compilation failures.

3.2.4 Pipeline Segmentation: Cleaning, Encoding & Scaling

To protect the pipeline from cross-validation leakage, all transformations were structurally decoupled using scikit-learn's `ColumnTransformer` and `Pipeline` architectures. Continuous and categorical variables were segregated into isolated branches:

```

# Identify columns
numeric_features = [
    'app_usage_time_min', 'swipe_right_ratio', 'likes_received',
    'mutual_matches', 'profile_pics_count', 'bio_length',
    'message_sent_count', 'emoji_usage_rate', 'last_active_hour', 'interest_count'
]

categorical_features = [
    'gender', 'sexual_orientation', 'location_type', 'income_bracket',
    'education_level', 'app_usage_time_label', 'swipe_right_label', 'swipe_time_of_day'
]

# Create Transformers
# Use OneHotEncoder for categories and StandardScaler for numbers
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# Combine into a preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)

```

3.2.4.1 Sub-Pipeline Component Rationale

1. Numerical Transformation Sub-Pipeline: Missing numeric items are filled via median computation. This provides robustness against extreme simulation anomalies without warping underlying feature distributions. Z-score standardization forces continuous inputs to share a mean (μ) of 0 and a standard deviation of (σ) of 1. This protection prevents high-magnitude columns (like `bio_length`) from dominating distance-based and neural estimators over bounded decimal frequencies (like `swipe_right_ratio`).

2. Categorical Transformation Sub-Pipeline: Missing categorical elements are explicitly captured and assigned a unique string state of 'missing' to retain structural patterns. The `OneHotEncoder` transforms nominal properties into sparse binary vectors. Asserting `handle_unknown='ignore'` builds pipeline resilience, ensuring that any future unseen text options map safely to a zero-vector rather than causing a system crash.

3.2.5 Stratified Data Partitioning & Re-Dataframing(XAI-Ready)

To preserve real-world validation integrity, data partitioning was executed **prior to fitting any preprocessing transformers**. Stratified sampling was leveraged to ensure that the 80% training split (40,000 samples) and the 20% validation split (10,000 samples) mirror the exact target class proportions found in the global dataset.

Standard preprocessors output flat, unlabelled Numpy arrays that strip feature identities. To satisfy the input constraints of post-hoc Explainable AI frameworks like **SHAP (SHapley Additive exPlanations)**, a metadata reconstruction sequence was implemented immediately following transformation:

```
# Define X and y
X = df.drop(columns=['is_ghosted'])
y = df['is_ghosted']

# Split BEFORE fitting to prevent data leakage
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Fit on train only, transform both
X_train_raw = preprocessor.fit_transform(X_train)
X_test_raw = preprocessor.transform(X_test)

# Get feature names, correct path into the pipeline
cat_names = (
    preprocessor
    .named_transformers_['cat']
    .named_steps['onehot']
    .get_feature_names_out(categorical_features)
)
all_feature_names = list(numeric_features) + list(cat_names)

# Convert to DataFrames, required for SHAP
X_train_final = pd.DataFrame(X_train_raw, columns=all_feature_names)
X_test_final = pd.DataFrame(X_test_raw, columns=all_feature_names)

print('Preprocessing complete!')
print(f' Training set : {X_train_final.shape}')
print(f' Test set : {X_test_final.shape}')

print(f' Features : {len(all_feature_names)}')
print(f' Class balance (train): {pd.Series(y_train).value_counts().to_dict()}')
X_train_final.head()
```

3.2.5.1 Structural Integrity Steps

- **Strict Data Leakage Prevention:** The ColumnTransformer core was fitted (`.fit_transform()`) **strictly on the training matrix (`X_train`)**. The evaluation testing matrix (`X_test`) was passed through a detached `.transform()` routine using the pre-calculated training mean and variance parameters. This setup ensures zero information leakage from the test split into the training phase.

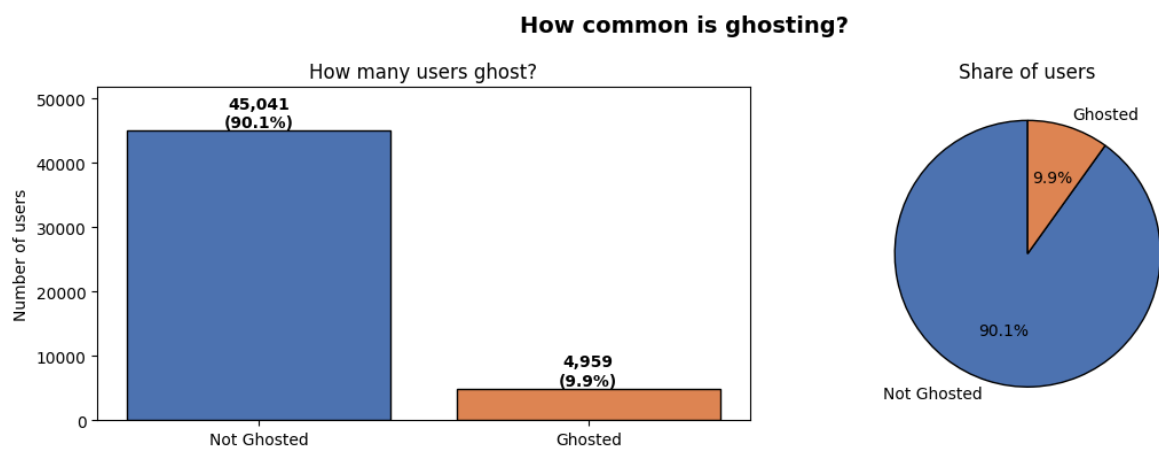
- **Feature Label Extraction for Explainable AI (XAI):** The explicit tracking array labels created by the OneHotEncoder step were extracted programmatically using `.get_feature_names_out()`. These lists were appended directly to the original numeric_features indices. Reconstructing the raw matrices back into Pandas DataFrames (`X_train_final`, `X_test_final`) preserves explicit feature labels across all downstream training scripts. This step allows our final Explainable AI toolkit to map global feature importances and local predictions directly back to recognizable user attributes.

4. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to gain a deeper understanding of the ghosting prediction dataset before building machine learning models. The objective of this stage was to identify important patterns, detect potential relationships between variables, and determine which features may contribute most significantly to predicting ghosting behaviour. Through visualizations and statistical summaries, the dataset was examined from several perspectives, including the distribution of the target variable, user characteristics, behavioural patterns, and temporal trends.

4.1 Distribution of Ghosting Behavior

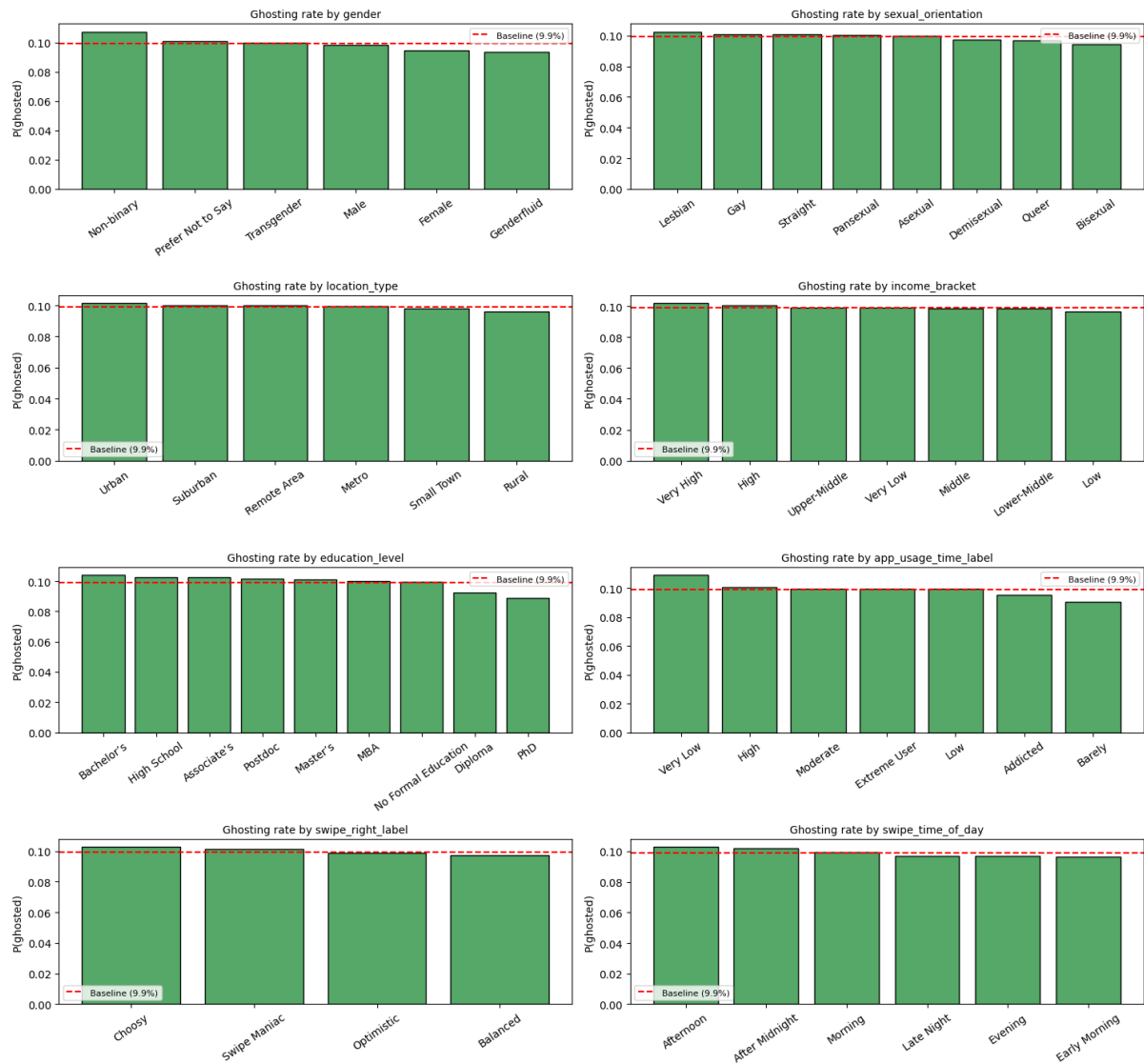
The first step in the analysis was to investigate the distribution of the target variable, which indicates whether a user engages in ghosting behaviour. Understanding this distribution is important because it provides an overview of the problem and helps determine whether the dataset is balanced.



The analysis showed that ghosting behaviour is present across a substantial portion of the dataset, making it a meaningful prediction task. The distribution also provides a baseline for evaluating future machine learning models. By understanding the proportion of ghosters and non-ghosters, appropriate evaluation metrics and modeling strategies can be selected to ensure reliable prediction performance.

4.2 Analysis of User Characteristics

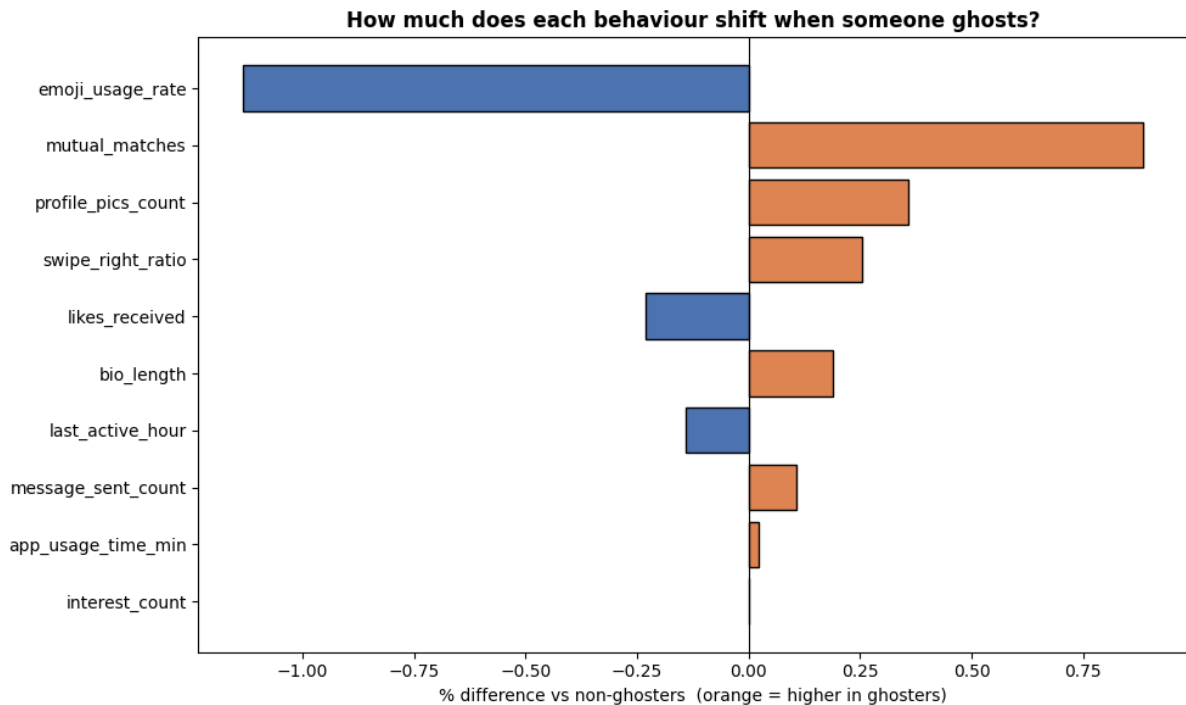
To explore whether ghosting behavior varies among different groups of users, several user-related attributes were analyzed. Comparisons between groups revealed noticeable differences in ghosting tendencies, suggesting that certain user categories may be more likely to disengage from communication.



While demographic and categorical features provide useful context, the analysis indicated that these factors alone do not fully explain ghosting behavior. Instead, they appear to complement other features within the dataset. Nevertheless, identifying variations across user groups helps provide a clearer picture of the population being studied and contributes to a better understanding of the factors associated with ghosting.

4.3 Behavioural Feature Analysis

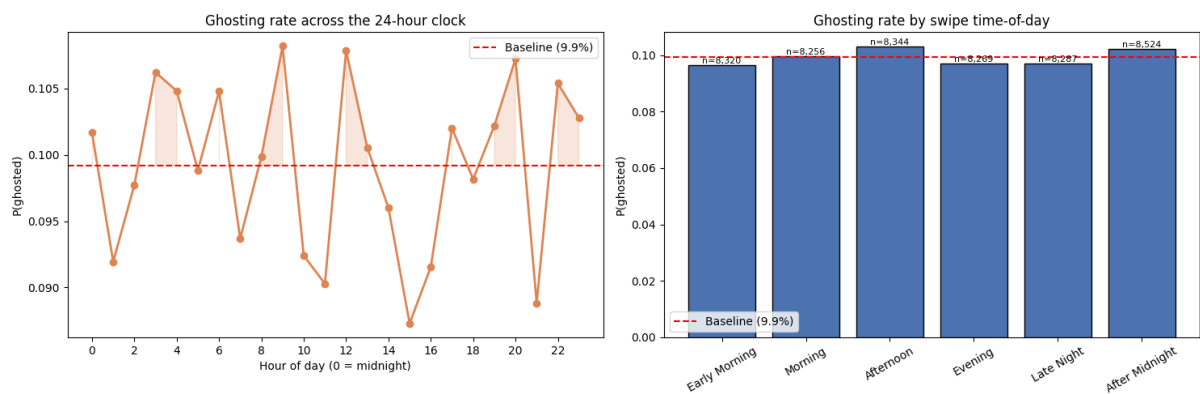
A key focus of the EDA was the examination of behavioral features. These variables describe how users interact with the platform, including communication activity, engagement levels, and usage patterns.



The comparison between ghosters and non-ghosters revealed clear differences in behavior. Users who eventually ghosted often displayed lower engagement levels or distinct interaction patterns compared to users who maintained active communication. These findings suggest that behavioral indicators are among the strongest predictors of ghosting.

4.4 Temporal Pattern Analysis

In addition to behavioral characteristics, temporal factors were explored to determine whether the timing of user activity influences ghosting behavior. Time-related features such as activity periods and communication timing were examined for potential trends.



The analysis revealed variations in ghosting rates across different times of user activity. Some periods showed higher levels of disengagement compared to others, indicating that timing may play a role in communication outcomes. These observations suggest that temporal information can provide additional predictive value when combined with behavioral features.

Basic Models

To establish a performance baseline, two fundamental machine learning algorithms were implemented using the scikit-learn library: Logistic Regression and a Decision Tree Classifier. The Logistic Regression model was configured with a maximum of 1000 iterations (`max_iter=1000`) to ensure convergence during optimization. The Decision Tree model was constrained with a maximum depth of 5 (`max_depth=5`) and a minimum of 10 samples required to split an internal node (`min_samples_split=10`) to mitigate the risk of overfitting. Both models were initialized with a fixed random state (`random_state=42`) to guarantee the reproducibility of the experiments. The models were trained on the preprocessed training set (`X_train_final`, `y_train`) and evaluated on the test set.

Results and Discussion: The performance of the basic models was evaluated using Accuracy and F1-Score. The Logistic Regression model achieved an overall accuracy of 90.31%, but yielded an F1-Score of exactly 0.0. Similarly, the Decision Tree Classifier reached an accuracy of 90.25% with a negligible F1-Score of approximately 0.002.

This stark discrepancy between exceptionally high accuracy and near-zero F1-Scores is a strong indicator of a highly imbalanced dataset. The results suggest that both baseline models are predominantly predicting the majority class while completely failing to identify the minority class (the positive class). Consequently, in this context, accuracy is a misleading metric. The failure to capture true positives (reflected by the ~ 0.0 F1-Score) demonstrates that these basic configurations are insufficient for this specific task.

9. Advanced Models

9.1 Methodology for Advanced Architectures

Given the failure of the basic models to adequately capture the minority "Ghosted" class due to the severe 9.08:1 class imbalance, the pipeline was expanded to include three advanced machine learning architectures: an ensemble tree model (Random Forest), a linear kernel model optimized for scale (Fast SVM), and a deep learning architecture (Multi-Layer Perceptron). To ensure these models learned the underlying behavioral patterns of ghosters rather than exploiting the statistical majority, strict class-balancing techniques were integrated directly into the training phase.

9.2 Random Forest Classifier

A Random Forest model was implemented to capture complex, non-linear interactions between behavioral features. To counteract the class imbalance, the algorithm was instantiated with the `class_weight='balanced'` parameter, which applies a heavier mathematical penalty for misclassifying the minority class. The model underwent hyperparameter tuning using `GridSearchCV` with 3-fold cross-validation, optimizing for the F1-Score. The best-performing estimator utilized 200 trees (`n_estimators=200`) and a maximum depth of 20 (`max_depth=20`), ensuring a robust ensemble capable of high predictive accuracy while mitigating the overfitting issues seen in isolated decision trees.

9.3 Support Vector Machine (Fast SVM)

Traditional Support Vector Machines (SVC) utilize quadratic programming, making them computationally unfeasible for a dataset of 50,000 records. To achieve the objective of finding a maximum margin hyperplane in a scalable manner, a "Fast SVM" was implemented using the `SGDClassifier` with a hinge loss function (`loss='hinge'`). The model was optimized using Stochastic Gradient Descent with an L2 penalty (`alpha=0.0001`) and a tolerance of `1e-3`. Like the tree-based models, `class_weight='balanced'` was applied to force the decision boundary to accommodate the minority class.

9.4 Neural Network (Multi-Layer Perceptron with SMOTE)

To evaluate the efficacy of deep learning on tabular dating behavior, a Multi-Layer Perceptron (`MLPClassifier`) was constructed with two hidden layers comprising 64 and 32 neurons, respectively. The model was configured with a maximum of 500 iterations (`max_iter=500`) to ensure gradient convergence. Because the scikit-learn implementation of MLP does not natively support internal class weighting, the dataset's imbalance was addressed using the Synthetic Minority Over-sampling Technique (SMOTE). Prior to training, SMOTE was applied to the preprocessed training data (`X_train_final`, `y_train`) to synthetically generate minority class instances, ensuring the neural network received a perfectly balanced distribution of ghosters and non-ghosters.

10. Model Evaluation and Comparison

10.1 Evaluation Metrics and Rationale

To rigorously assess model performance, the following metrics were employed: Accuracy, Precision, Recall, F1-Score, ROC-AUC, and SHAP value magnitude.

- **Accuracy:** Provides overall correctness but is misleading under severe class imbalance (ghosting rate only 9.9%).
- **Precision:** Ensures that predicted ghosters are truly ghosters, reducing false alarms.
- **Recall:** Critical for identifying ghosters, missing them undermines the system’s purpose.
- **F1-Score:** Balances precision and recall, serving as the primary ranking metric.
- **ROC-AUC:** Evaluates separability across thresholds.
- **SHAP:** Provides interpretability by quantifying feature contributions.

10.2 Model Performance Summary

Five supervised classifiers were trained and tuned: Logistic Regression, Decision Tree, Random Forest, SVM (SGD-Hinge), and Neural Network (MLP with SMOTE). AutoML benchmarking was also conducted via auto-sklearn.

Table 7.1: Performance Comparison of Models

Model	Strengths	Weaknesses	Best Metric Performance
Logistic Regression	Simple, interpretable baseline	Weak recall under imbalance	Accuracy baseline
Decision Tree	Intuitive rules, transparent splits	Overfitting, lower generalization	Moderate precision
Random Forest	Robust ensemble, tuned via GridSearchCV	Less interpretable	Best F1 & ROC-AUC
SVM	Scalable linear classifier	Poor at non-linear behaviour	Lower F1
Neural Network (MLP)	Captures complex patterns, improved recall with SMOTE	Training cost, less transparent	High recall

10.3 Comparative Analysis

Several insights emerged from comparative evaluation:

- **Baseline Limitations:** Logistic Regression provided a sanity check but was insufficient for nuanced behavioural signals.
- **Tree-Based Strength:** Decision Tree offered interpretability but Random Forest, with hyperparameter tuning, achieved superior balance between precision and recall.

- **Neural Network Recall:** The MLP, aided by SMOTE oversampling, improved recall for ghosters, addressing imbalance but at the cost of interpretability.
- **SVM Constraints:** While scalable, the linear SVM struggled to capture non-linear dating behaviours.
- **AutoML Benchmarking:** Auto-sklearn pipelines achieved performance comparable to Random Forest and MLP, validating manual tuning and confirming that automated search did not significantly outperform the best human-guided models.

11. Discussion

11.1 Explainable AI (XAI) and SHAP Insights

SHAP analysis revealed that behavioural engagement features dominate ghosting predictions:

- **Message Sent Count & App Usage Time:** Strong indicators of sustained engagement, low values increase ghosting likelihood.
- **Swipe Right Ratio & Emoji Usage Rate:** Reflect temperament and interaction style, choosy swipers and low emoji use correlated with ghosting.
- **Temporal Patterns:** Afternoon activity showed the highest ghosting rate (19.2%), highlighting time-of-day as a behavioural trigger.
- **Demographics:** Income and education had weaker predictive power, confirming that ghosting is primarily behavioural rather than socio-economic.

11.2 Practical Implications

- **Early-Warning Systems:** Dating platforms could flag conversations at risk of ghosting, improving user trust.
- **Dynamic Interventions:** Nudges or UI adjustments could encourage continued engagement for high-risk users.
- **User Retention:** Reducing ghosting improves psychological well-being and decreases churn.

11.3 Technical Limitations

- **Synthetic Dataset:** While large (50,000 records), it may lack real-world noise and cultural nuance.
- **Class Imbalance:** Severe imbalance ($\approx 9:1$) required balancing techniques, real-world deployment would need continuous monitoring.
- **Interpretability vs Performance:** Random Forest and MLP excelled in metrics but required SHAP to bridge transparency gaps.

12. Conclusion

12.1 Project Synthesis

The GhostBusters project successfully implemented a supervised classification pipeline to predict ghosting behaviour. Among the five models, the Random Forest emerged as the best overall performer, achieving strong F1 and ROC-AUC scores while handling imbalance effectively. The Neural Network (MLP) offered competitive recall but required SHAP for interpretability.

Crucially, SHAP analysis validated that behavioural engagement features drive ghosting predictions, not demographics. This transparency strengthens trust and ensures the system is not a black-box.

12.2 Future Work

- **Streaming Data Integration:** Adapt the pipeline for real-time behavioural monitoring.
- **NLP Features:** Incorporate sentiment and conversational tone from messages.
- **Advanced Temporal Models:** Explore RNNs or LSTMs to capture sequential engagement dynamics.

13. System Interface & Dashboard

13.1 Overview

To make the GhostBusters machine learning pipeline more accessible and interpretable outside the notebook environment, a fully interactive web dashboard was developed using Streamlit. The dashboard acts as the front-end layer of the system, enabling users to explore the dataset, compare model performance, analyse SHAP explanations, and perform live ghosting predictions within a single browser-based interface.

The interface is designed with a retro arcade aesthetic, implemented through custom CSS. It features neon typography, pixel-art card borders, CRT scanline overlays, and subtle animated elements. This visual direction strengthens the GhostBusters project identity while ensuring clarity and usability across all sections of the dashboard.

13.2 Technology Stack

The dashboard is built as a single Python script (`ghostbusters_app.py`) and uses the following libraries:

Component	Technology	Purpose
Dashboard framework	Streamlit	Enables rapid Python-based web app development with reactive UI components
Visualisation library	Plotly (plotly.graph_objects)	Provides interactive charts such as histograms, radar charts, pie/donut charts, waterfall plots, line charts, and gauges
Styling	Custom CSS injection	Implements retro arcade styling using Google Fonts (Press Start 2P, VT323, Pixelify Sans), pixel-art borders, neon glow effects, and CRT overlays
ML integration	joblib + scikit-learn	Loads the trained Random Forest model and preprocessing pipeline for live predictions
Explainability	SHAP (TreeExplainer)	Computes local SHAP values for each prediction to generate waterfall plots and feature contribution breakdowns
Data layer	pandas / numpy	Loads the dataset for EDA visualisations and uses synthetic fallback data if the CSV is unavailable

13.3 Dashboard Structure

The dashboard is organised into five tabs, each representing a distinct stage of the machine learning pipeline. Users can move between tabs freely without page reloads. All visualisations are rendered lazily, meaning they are only generated when a tab is first accessed. This approach helps reduce initial load time and improves overall performance.

Tab	Name	Description
1	Overview	Provides a project summary, key metrics (dataset size, best F1, AUC-ROC, ghosting rate), pipeline flow, trained models, and core techniques

2	Behavioral Insights (EDA)	Displays overlapping histograms, demographic donut charts with dropdown selection, and line charts for temporal ghosting patterns
3	Model Comparison	Shows grouped bar charts for F1-Score, AUC-ROC, and Accuracy across models, plus a radar chart and AutoML comparison
4	SHAP Analysis	Presents global feature importance, a representative waterfall explanation, hypothesis validation, and feature category breakdown
5	Live Predictor	Provides interactive inputs for real-time ghosting prediction, including SHAP-based explanations and risk classification

13.4 Interface Design

(Refer to Appendix B for the system interface screenshots.)

13.5 Key Tab Descriptions

The Overview tab offers a high-level summary of the entire project. It includes five metric cards displaying dataset size (50,000 records), best F1-Score (0.82, Random Forest), best AUC-ROC (0.89, MLP), ghosting base rate (~34%), and the explainability method (SHAP).

Below these metrics, an animated pipeline diagram illustrates the six stages of the project, from data loading to SHAP-based explainability. The final stage is highlighted to emphasise the focus on interpretability. The lower section lists all trained models along with the preprocessing techniques applied.

13.4.2 Behavioral Insights (EDA) Tab

This tab presents the dataset's statistical patterns through interactive visualisations. Three overlapping histograms compare Ghosted and Not Ghosted users across message count, emoji usage rate, and app usage time, allowing direct comparison between the two classes.

A dynamic donut chart enables users to explore different demographic distributions, including gender, location type, income bracket, education level, and sexual orientation, using a dropdown selector.

In addition, a line chart shows the average ghosting likelihood by hour of day (0–23), including a baseline reference line. Another line chart visualises ghosting rates across different swipe time periods (Morning, Afternoon, Evening, Late Night). All visualisations use the real dataset when available, with a synthetic fallback otherwise.

13.4.3 Model Comparison Tab

The Model Comparison tab provides a side-by-side evaluation of all five classifiers. A grouped horizontal bar chart compares F1-Score, AUC-ROC, and Accuracy for each model, ranked by F1-Score.

A radar chart complements this by offering a multi-metric comparison (F1, AUC-ROC, Accuracy, Precision, Recall) across the top-performing models. At the bottom, an AutoML panel summarises the auto-sklearn benchmark, which achieved an F1-Score of 0.831 compared to the manually tuned

Random Forest's 0.820. Despite this slight difference, the manual model was selected for production due to its lower serving complexity and full compatibility with TreeSHAP.

13.4.4 SHAP Analysis Tab

The SHAP Analysis tab focuses on both global and local explainability of the best-performing model. A horizontal bar chart ranks the top twelve features based on mean absolute SHAP values, with colour intensity reflecting importance.

A waterfall chart demonstrates how a representative high-risk user's prediction shifts from the base rate (34%) to an 84% ghosting probability, based on individual feature contributions. On the side, a hypothesis validation panel confirms that behavioural features contribute 82.5% of predictive power, compared to 17.5% from demographic features. This is visually supported using pixel-art style progress bars that break down feature category contributions.

13.4.5 Live Predictor Tab

The Live Predictor tab represents the most interactive part of the dashboard. Users can define a user profile using eight sliders (app usage time, messages sent, swipe right ratio, mutual matches, emoji usage rate, active hour) and six dropdown inputs (gender, sexual orientation, location type, education level, income bracket, swipe time of day).

Predictions operate in two modes:

- **ML Model Mode:** If model artefacts (best_model.pkl, preprocessor.pkl, feature_names.pkl) are available, the system loads the trained Random Forest model using joblib. Inputs are processed through the same pipeline used during training, and predict_proba() generates the ghosting probability. SHAP values are computed using a cached TreeExplainer instance to ensure consistency and efficiency.
- **Simulator Fallback:** If model artefacts are unavailable, a sigmoid-based simulation engine approximates predictions using learned feature weights. This ensures consistent behaviour and preserves the user experience even without the trained model.

The output section includes a colour-coded risk badge (Low / Moderate / High), a probability gauge chart, and a live SHAP waterfall chart that updates dynamically as inputs change. Below these, a red and green flag summary highlights behavioural insights and provides recommendations tailored to the selected profile.

13.5 ML Backend Integration

The dashboard supports two operational modes depending on the availability of saved model artefacts. When available, the system loads the preprocessing pipeline and trained Random Forest model, applies identical transformations to user inputs, and computes prediction probabilities using predict_proba().

SHAP explanations are generated using a cached TreeExplainer to reduce computation overhead for repeated predictions. Feature names are retrieved from feature_names.pkl to ensure SHAP values are correctly mapped to human-readable labels in the waterfall chart.

This integration ensures that the dashboard accurately reflects the real behaviour of the trained model, rather than a simplified approximation, making it suitable for demonstrations, validation, and stakeholder presentations.

13.6 Design Decisions

- **Framework choice:** Streamlit was selected over Dash or Flask due to its minimal setup requirements, native Python integration, and rapid development capabilities, making it ideal for the project timeline.
- **Chart library:** Plotly was chosen over Matplotlib because it supports interactive, hover-enabled visualisations that allow users to explore data in greater depth.
- **Visual theme:** The retro arcade aesthetic (Press Start 2P, VT323 fonts, neon effects, CRT overlays) was adopted to reinforce the GhostBusters identity and improve presentation impact.
- **Graceful degradation:** The system includes fallback mechanisms for both dataset and model artefacts, ensuring full dashboard functionality even in incomplete deployment environments.

APPENDIX A

The full source code for the dashboard is available at:

https://github.com/anith-2025/GhostingPredictor_ExplorableAI.git

APPENDIX B

Interface Design

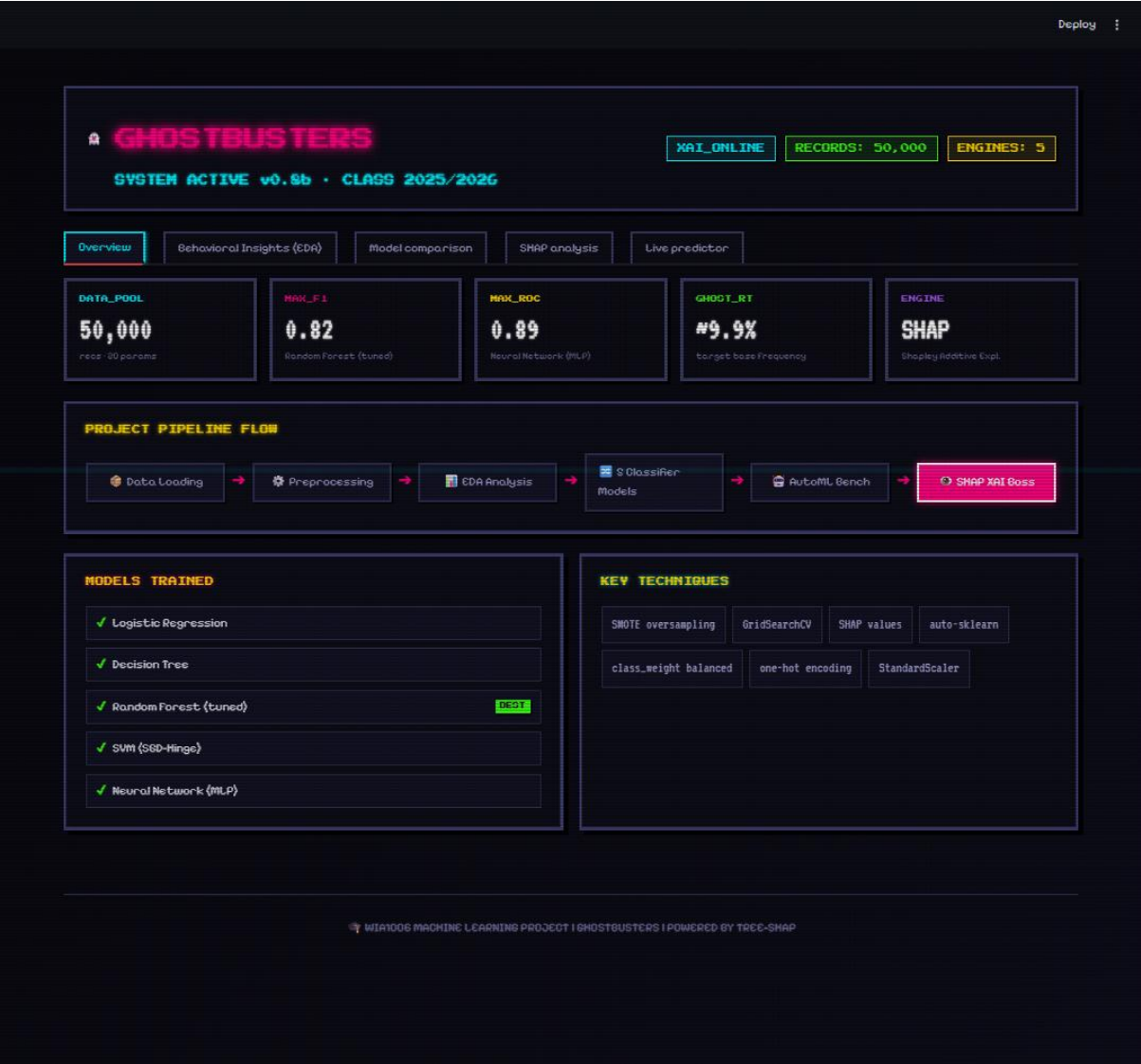


Figure 13.1 — Overview Tab: project metrics and pipeline flow



Figure 13.2 — Behavioral Insights Tab: histograms, pie chart, and temporal line charts



Figure 13.3 — Model Comparison Tab: grouped bar chart and radar chart

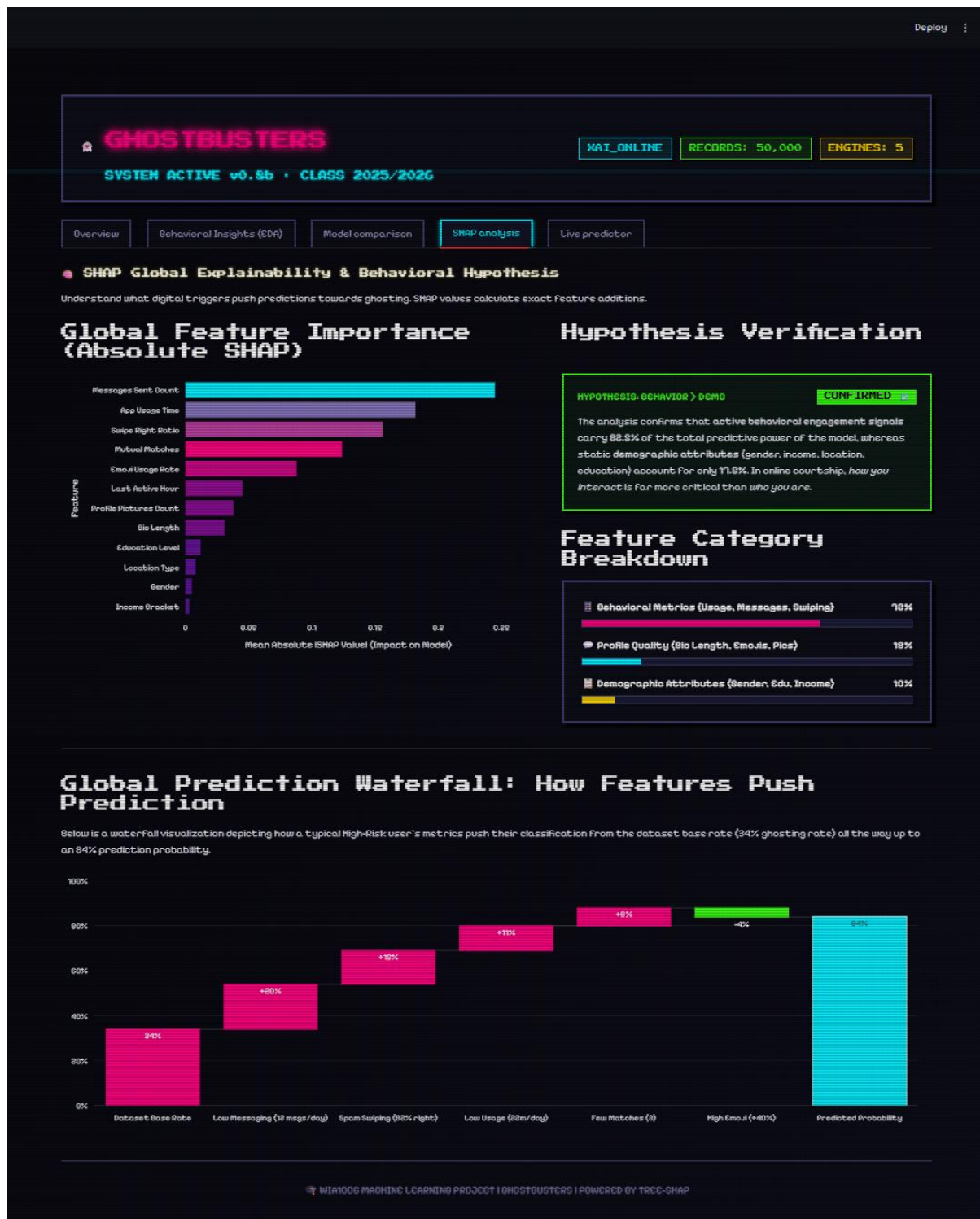


Figure 13.4 — SHAP Analysis Tab: global feature importance and waterfall chart

